```python
import numpy as np
import pandas as pd

# Load dataset
data = pd.read_csv("plant_stress_data.csv")

# ----------------------------
# DATA CLEANING
# ----------------------------

# Convert Rain column (if categorical)
if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({
        "No Rain": 0,
        "Rain": 1
    })

# Convert all columns to numeric
cols = ["Temperature", "Humidity", "Soil Moisture", "Rain", "Stress"]

for col in cols:
    data[col] = pd.to_numeric(data[col], errors='coerce')

# Remove missing values
data = data.dropna()

# ----------------------------
# EXTRACT FEATURES
# ----------------------------
T = data["Temperature"].values
H = data["Humidity"].values
S = data["Soil Moisture"].values
R = data["Rain"].values
y = data["Stress"].values

# ----------------------------
# HYPERPARAMETERS
# ----------------------------
T_opt = 25
T_range = 15
learning_rate = 0.01
epochs = 200
decay = 0.5

# Initial weights
w = np.array([0.3, 0.3, 0.25, 0.15])

# Memory buffer
memory = []

# Sigmoid function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# ----------------------------
# TRAINING LOOP
# ----------------------------
for epoch in range(epochs):

    total_error = 0

    for i in range(len(T)):

        # Normalize features
        Ts = abs(T[i] - T_opt) / T_range   # improved temperature stress
        Hs = H[i] / 100
        Ss = 1 - (S[i] / 1023)
        Rs = R[i]

        features = np.array([Ts, Hs, Ss, Rs])

        # Linear stress score
        stress = np.dot(w, features)

        # Memory mechanism
        memory.append(stress)
        if len(memory) > 5:
            memory.pop(0)

        cumulative = 0
        for j, val in enumerate(reversed(memory)):
            cumulative += val * np.exp(-decay * j)

        # Prediction
        pred = sigmoid(cumulative)
```

```
        # Error
        error = pred - y[i]
        total_error += error ** 2

        # Weight update (gradient descent)
        w = w - learning_rate * error * features

    print(f"Epoch {epoch+1}, Error: {total_error:.4f}")

# ----------------------------
# FINAL OUTPUT
# ----------------------------
print("\nTrained Weights:", w)
```

```
Epoch 1, Error: 67.8324
Epoch 2, Error: 66.8280
Epoch 3, Error: 66.5539
Epoch 4, Error: 66.3288
Epoch 5, Error: 66.1164
Epoch 6, Error: 65.9121
Epoch 7, Error: 65.7148
Epoch 8, Error: 65.5239
Epoch 9, Error: 65.3389
Epoch 10, Error: 65.1595
Epoch 11, Error: 64.9854
Epoch 12, Error: 64.8163
Epoch 13, Error: 64.6520
Epoch 14, Error: 64.4922
Epoch 15, Error: 64.3368
Epoch 16, Error: 64.1856
Epoch 17, Error: 64.0384
Epoch 18, Error: 63.8951
Epoch 19, Error: 63.7555
Epoch 20, Error: 63.6195
Epoch 21, Error: 63.4869
Epoch 22, Error: 63.3577
Epoch 23, Error: 63.2317
Epoch 24, Error: 63.1087
Epoch 25, Error: 62.9888
Epoch 26, Error: 62.8718
Epoch 27, Error: 62.7576
Epoch 28, Error: 62.6461
Epoch 29, Error: 62.5372
Epoch 30, Error: 62.4309
Epoch 31, Error: 62.3270
Epoch 32, Error: 62.2256
Epoch 33, Error: 62.1264
Epoch 34, Error: 62.0295
Epoch 35, Error: 61.9347
Epoch 36, Error: 61.8421
Epoch 37, Error: 61.7515
Epoch 38, Error: 61.6629
Epoch 39, Error: 61.5763
Epoch 40, Error: 61.4915
Epoch 41, Error: 61.4085
Epoch 42, Error: 61.3274
Epoch 43, Error: 61.2479
Epoch 44, Error: 61.1701
Epoch 45, Error: 61.0940
Epoch 46, Error: 61.0194
Epoch 47, Error: 60.9463
Epoch 48, Error: 60.8748
Epoch 49, Error: 60.8048
Epoch 50, Error: 60.7361
Epoch 51, Error: 60.6689
Epoch 52, Error: 60.6030
Epoch 53, Error: 60.5384
Epoch 54, Error: 60.4751
Epoch 55, Error: 60.4131
Epoch 56, Error: 60.3522
Epoch 57, Error: 60.2926
Epoch 58, Error: 60.2342
```

```
import numpy as np
import pandas as pd

# ----------------------------
# LOAD DATA
# ----------------------------
data = pd.read_csv("plant_stress_data.csv")

# ----------------------------
# DATA CLEANING
# ----------------------------
if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({
        "No Rain": 0,
        "Rain": 1
    })

cols = ["Temperature", "Humidity", "Soil Moisture", "Rain", "Stress"]

for col in cols:
```

```
    data[col] = pd.to_numeric(data[col], errors='coerce')

data = data.dropna()

# ----------------------------
# FEATURES
# ----------------------------
T = data["Temperature"].values
H = data["Humidity"].values
S = data["Soil Moisture"].values
R = data["Rain"].values
y = data["Stress"].values

# ----------------------------
# TRAIN-TEST SPLIT (85-15)
# ----------------------------
split = int(0.85 * len(T))

T_train, T_test = T[:split], T[split:]
H_train, H_test = H[:split], H[split:]
S_train, S_test = S[:split], S[split:]
R_train, R_test = R[:split], R[split:]
y_train, y_test = y[:split], y[split:]

# ----------------------------
# HYPERPARAMETERS
# ----------------------------
T_opt = 25
T_range = 15
learning_rate = 0.01
epochs = 200
decay = 0.5

# Initial weights
w = np.array([0.3, 0.3, 0.25, 0.15])

# ----------------------------
# FUNCTIONS
# ----------------------------
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def compute_features(T, H, S, R):
    Ts = abs(T - T_opt) / T_range
    Hs = H / 100
    Ss = 1 - (S / 1023)
    Rs = R
    return np.array([Ts, Hs, Ss, Rs])

def forward_pass(features, w, memory):
    stress = np.dot(w, features)

    memory.append(stress)
    if len(memory) > 5:
        memory.pop(0)

    cumulative = 0
    for j, val in enumerate(reversed(memory)):
        cumulative += val * np.exp(-decay * j)

    pred = sigmoid(cumulative)
    return pred

# ----------------------------
# TRAINING
# ----------------------------
for epoch in range(epochs):

    total_error = 0
    memory = []

    for i in range(len(T_train)):

        features = compute_features(
            T_train[i], H_train[i], S_train[i], R_train[i]
        )

        pred = forward_pass(features, w, memory)

        error = pred - y_train[i]
        total_error += error ** 2

        # Gradient update
        w = w - learning_rate * error * features

    print(f"Epoch {epoch+1}, Error: {total_error:.4f}")

print("\n✅ Trained Weights:", w)
```

```python
# ----------------------------
# EVALUATION
# ----------------------------
def evaluate(T, H, S, R, y, w):

    correct = 0
    memory = []

    for i in range(len(T)):

        features = compute_features(T[i], H[i], S[i], R[i])
        pred = forward_pass(features, w, memory)

        predicted_label = 1 if pred > 0.5 else 0

        if predicted_label == y[i]:
            correct += 1

    accuracy = correct / len(T)
    return accuracy

train_acc = evaluate(T_train, H_train, S_train, R_train, y_train, w)
test_acc = evaluate(T_test, H_test, S_test, R_test, y_test, w)

print(f"\n📊 Train Accuracy: {train_acc:.2f}")
print(f"📊 Test Accuracy: {test_acc:.2f}")

# ----------------------------
# TESTING ON NEW DATA
# ----------------------------
def predict_stress(T, H, S, R, w):

    memory = []

    features = compute_features(T, H, S, R)
    pred = forward_pass(features, w, memory)

    return pred

# Example test
sample_pred = predict_stress(30, 60, 500, 0, w)

print("\n🌿 Sample Prediction:", sample_pred)
print("Status:", "STRESSED" if sample_pred > 0.5 else "HEALTHY")

# ----------------------------
# REAL-TIME (ESP8266 SIMULATION)
# ----------------------------
def real_time_system(sensor_data, w):

    memory = []

    for reading in sensor_data:

        T, H, S, R = reading

        pred = predict_stress(T, H, S, R, w)

        print("\nSensor Data:", reading)
        print("Stress Probability:", round(pred, 3))

        if pred > 0.5:
            print("⚠️ ALERT: Plant is STRESSED → Turn ON irrigation")
        else:
            print("✅ Plant is Healthy")

# Simulated live data
sensor_stream = [
    (27, 59, 560, 0),
    (32, 55, 400, 0),
    (25, 70, 700, 1)
]

real_time_system(sensor_stream, w)
```

```
Epoch 1, Error: 57.0364
Epoch 2, Error: 56.3512
Epoch 3, Error: 56.1590
Epoch 4, Error: 56.0217
Epoch 5, Error: 55.8951
Epoch 6, Error: 55.7727
Epoch 7, Error: 55.6533
Epoch 8, Error: 55.5363
Epoch 9, Error: 55.4219
Epoch 10, Error: 55.3097
Epoch 11, Error: 55.1999
Epoch 12, Error: 55.0924
Epoch 13, Error: 54.9871
```

```
Epoch 14, Error: 54.8840
Epoch 15, Error: 54.7830
Epoch 16, Error: 54.6841
Epoch 17, Error: 54.5873
Epoch 18, Error: 54.4925
Epoch 19, Error: 54.3996
Epoch 20, Error: 54.3087
Epoch 21, Error: 54.2196
Epoch 22, Error: 54.1324
Epoch 23, Error: 54.0470
Epoch 24, Error: 53.9633
Epoch 25, Error: 53.8814
Epoch 26, Error: 53.8011
Epoch 27, Error: 53.7225
Epoch 28, Error: 53.6455
Epoch 29, Error: 53.5700
Epoch 30, Error: 53.4961
Epoch 31, Error: 53.4237
Epoch 32, Error: 53.3528
Epoch 33, Error: 53.2832
Epoch 34, Error: 53.2151
Epoch 35, Error: 53.1483
Epoch 36, Error: 53.0829
Epoch 37, Error: 53.0187
Epoch 38, Error: 52.9558
Epoch 39, Error: 52.8942
Epoch 40, Error: 52.8337
Epoch 41, Error: 52.7745
Epoch 42, Error: 52.7164
Epoch 43, Error: 52.6594
Epoch 44, Error: 52.6036
Epoch 45, Error: 52.5488
Epoch 46, Error: 52.4950
Epoch 47, Error: 52.4423
Epoch 48, Error: 52.3906
Epoch 49, Error: 52.3399
Epoch 50, Error: 52.2901
Epoch 51, Error: 52.2413
Epoch 52, Error: 52.1933
Epoch 53, Error: 52.1463
Epoch 54, Error: 52.1002
Epoch 55, Error: 52.0549
Epoch 56, Error: 52.0105
Epoch 57, Error: 51.9669
```

```
import numpy as np
import pandas as pd

# -----------------------------
# LOAD + CLEAN DATA
# -----------------------------
data = pd.read_csv("plant_stress_data.csv")

if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({"No Rain": 0, "Rain": 1})

cols = ["Temperature", "Humidity", "Soil Moisture", "Rain", "Stress"]
for col in cols:
    data[col] = pd.to_numeric(data[col], errors='coerce')

data = data.dropna()

# -----------------------------
# FEATURES
# -----------------------------
T = data["Temperature"].values
H = data["Humidity"].values
S = data["Soil Moisture"].values
R = data["Rain"].values
y = data["Stress"].values

# -----------------------------
# TRAIN-TEST SPLIT
# -----------------------------
split = int(0.85 * len(T))

T_train, T_test = T[:split], T[split:]
H_train, H_test = H[:split], H[split:]
S_train, S_test = S[:split], S[split:]
R_train, R_test = R[:split], R[split:]
y_train, y_test = y[:split], y[split:]

# -----------------------------
# HYPERPARAMETERS
# -----------------------------
lr = 0.01
epochs = 700
decay = 0.6
momentum = 0.9

T_opt = 25
T_range = 15
```

```
# ----------------------------
# INITIAL WEIGHTS
# ----------------------------
w_main = np.random.rand(4)
w_inter = np.random.rand(3)
w_gate = np.random.rand(4)

v_main = np.zeros(4)
v_inter = np.zeros(3)
v_gate = np.zeros(4)

# ----------------------------
# FUNCTIONS
# ----------------------------
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def features(T, H, S, R):
    Ts = abs(T - T_opt) / T_range
    Hs = H / 100
    Ss = 1 - (S / 1023)
    Rs = R

    base = np.array([Ts, Hs, Ss, Rs])

    # Feature interactions
    inter = np.array([
        Ts * Ss,
        Hs * Ss,
        Ts * Hs
    ])

    return base, inter

# ----------------------------
# TRAINING
# ----------------------------
for epoch in range(epochs):

    memory = []
    total_loss = 0

    for i in range(len(T_train)):

        base, inter = features(T_train[i], H_train[i], S_train[i], R_train[i])

        instant = np.dot(w_main, base) + np.dot(w_inter, inter)

        # Memory
        memory.append(instant)
        if len(memory) > 6:
            memory.pop(0)

        mem_effect = sum(val * np.exp(-decay*j) for j, val in enumerate(reversed(memory)))

        # GATE
        gate = sigmoid(np.dot(w_gate, base))

        # FINAL OUTPUT
        combined = gate * mem_effect + (1 - gate) * instant
        pred = sigmoid(combined)

        error = pred - y_train[i]
        total_loss += error**2

        # ----------------------------
        # GRADIENT UPDATE + MOMENTUM
        # ----------------------------
        grad_main = error * base
        grad_inter = error * inter
        grad_gate = error * base

        v_main = momentum * v_main + lr * grad_main
        v_inter = momentum * v_inter + lr * grad_inter
        v_gate = momentum * v_gate + lr * grad_gate

        w_main -= v_main
        w_inter -= v_inter
        w_gate -= v_gate

    print(f"Epoch {epoch+1}, Loss: {total_loss:.4f}")

# ----------------------------
# EVALUATION
# ----------------------------
def evaluate(T, H, S, R, y):
```

```
    memory = []
    correct = 0

    for i in range(len(T)):

        base, inter = features(T[i], H[i], S[i], R[i])

        instant = np.dot(w_main, base) + np.dot(w_inter, inter)

        memory.append(instant)
        if len(memory) > 6:
            memory.pop(0)

        mem_effect = sum(val * np.exp(-decay*j) for j, val in enumerate(reversed(memory)))

        gate = sigmoid(np.dot(w_gate, base))
        combined = gate * mem_effect + (1 - gate) * instant

        pred = sigmoid(combined)
        label = 1 if pred > 0.5 else 0

        if label == y[i]:
            correct += 1

    return correct / len(T)

train_acc = evaluate(T_train, H_train, S_train, R_train, y_train)
test_acc = evaluate(T_test, H_test, S_test, R_test, y_test)

print("\n🔥 Improved Results")
print("Train Accuracy:", train_acc)
print("Test Accuracy:", test_acc)
```

```
Epoch 1, Loss: 57.9484
Epoch 2, Loss: 53.8148
Epoch 3, Loss: 52.3449
Epoch 4, Loss: 51.2452
Epoch 5, Loss: 50.3633
Epoch 6, Loss: 49.6225
Epoch 7, Loss: 48.9793
Epoch 8, Loss: 48.4086
Epoch 9, Loss: 47.8951
Epoch 10, Loss: 47.4291
Epoch 11, Loss: 47.0039
Epoch 12, Loss: 46.6145
Epoch 13, Loss: 46.2571
Epoch 14, Loss: 45.9285
Epoch 15, Loss: 45.6258
Epoch 16, Loss: 45.3465
Epoch 17, Loss: 45.0885
Epoch 18, Loss: 44.8499
Epoch 19, Loss: 44.6287
Epoch 20, Loss: 44.4235
Epoch 21, Loss: 44.2327
Epoch 22, Loss: 44.0550
Epoch 23, Loss: 43.8893
Epoch 24, Loss: 43.7345
Epoch 25, Loss: 43.5896
Epoch 26, Loss: 43.4538
Epoch 27, Loss: 43.3262
Epoch 28, Loss: 43.2063
Epoch 29, Loss: 43.0933
Epoch 30, Loss: 42.9866
Epoch 31, Loss: 42.8859
Epoch 32, Loss: 42.7905
Epoch 33, Loss: 42.7001
Epoch 34, Loss: 42.6143
Epoch 35, Loss: 42.5327
Epoch 36, Loss: 42.4551
Epoch 37, Loss: 42.3811
Epoch 38, Loss: 42.3105
Epoch 39, Loss: 42.2430
Epoch 40, Loss: 42.1785
Epoch 41, Loss: 42.1167
Epoch 42, Loss: 42.0575
Epoch 43, Loss: 42.0006
Epoch 44, Loss: 41.9460
Epoch 45, Loss: 41.8935
Epoch 46, Loss: 41.8429
Epoch 47, Loss: 41.7942
Epoch 48, Loss: 41.7472
Epoch 49, Loss: 41.7019
Epoch 50, Loss: 41.6581
Epoch 51, Loss: 41.6158
Epoch 52, Loss: 41.5749
Epoch 53, Loss: 41.5352
Epoch 54, Loss: 41.4968
Epoch 55, Loss: 41.4596
Epoch 56, Loss: 41.4235
Epoch 57, Loss: 41.3884
Epoch 58, Loss: 41.3544
```

```python
import numpy as np
import pandas as pd

# ----------------------------
# LOAD + CLEAN DATA
# ----------------------------
data = pd.read_csv("plant_stress_data.csv")

if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({"No Rain": 0, "Rain": 1})

cols = ["Temperature", "Humidity", "Soil Moisture", "Rain", "Stress"]

for col in cols:
    data[col] = pd.to_numeric(data[col], errors='coerce')

data = data.dropna()

# ----------------------------
# FEATURE SCALING (IMPORTANT)
# ----------------------------
T = data["Temperature"].values
H = data["Humidity"].values
S = data["Soil Moisture"].values
R = data["Rain"].values
y = data["Stress"].values

# Normalize everything properly
T = (T - np.mean(T)) / np.std(T)
H = H / 100
S = S / 1023

X = np.column_stack((T, H, S, R))

# ----------------------------
# TRAIN-TEST SPLIT
# ----------------------------
split = int(0.85 * len(X))

X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# ----------------------------
# MODEL 1: LOGISTIC REGRESSION
# ----------------------------
w_lr = np.zeros(4)
lr = 0.05
epochs = 300

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

for epoch in range(epochs):
    for i in range(len(X_train)):
        pred = sigmoid(np.dot(w_lr, X_train[i]))
        error = pred - y_train[i]
        w_lr -= lr * error * X_train[i]

# ----------------------------
# MODEL 2: MEMORY MODEL
# ----------------------------
w_mem = np.random.rand(4)
decay = 0.6

def memory_model(X):
    memory = []
    preds = []

    for x in X:
        stress = np.dot(w_mem, x)

        memory.append(stress)
        if len(memory) > 5:
            memory.pop(0)

        cumulative = sum(val * np.exp(-decay*j)
                         for j, val in enumerate(reversed(memory)))

        preds.append(sigmoid(cumulative))

    return np.array(preds)

# ----------------------------
# ENSEMBLE PREDICTION
# ----------------------------
def ensemble_predict(X):

    preds = []
```

```
    mem_preds = memory_model(X)

    for i in range(len(X)):
        lr_pred = sigmoid(np.dot(w_lr, X[i]))

        # Combine both models
        final_pred = 0.6 * lr_pred + 0.4 * mem_preds[i]

        preds.append(1 if final_pred > 0.5 else 0)

    return np.array(preds)

# ----------------------------
# EVALUATION
# ----------------------------
def accuracy(X, y):
    preds = ensemble_predict(X)
    return np.mean(preds == y)

train_acc = accuracy(X_train, y_train)
test_acc = accuracy(X_test, y_test)

print("\n🔥 FINAL IMPROVED MODEL")
print("Train Accuracy:", train_acc)
print("Test Accuracy:", test_acc)
```

```
🔥 FINAL IMPROVED MODEL
Train Accuracy: 0.6977611940298507
Test Accuracy: 0.6458333333333334
```

```
print("Class Distribution:")
print(pd.Series(y).value_counts(normalize=True))
```

```
Class Distribution:
1    0.686709
0    0.313291
Name: proportion, dtype: float64
```

```
import numpy as np
import pandas as pd

# ----------------------------
# LOAD + CLEAN DATA
# ----------------------------
data = pd.read_csv("plant_stress_data.csv")

if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({"No Rain": 0, "Rain": 1})

cols = ["Temperature", "Humidity", "Soil Moisture", "Rain", "Stress"]
for col in cols:
    data[col] = pd.to_numeric(data[col], errors='coerce')

data = data.dropna()

T = data["Temperature"].values
H = data["Humidity"].values
S = data["Soil Moisture"].values
R = data["Rain"].values
y = data["Stress"].values

# ----------------------------
# SPLIT
# ----------------------------
split = int(0.85 * len(T))

T_train, T_test = T[:split], T[split:]
H_train, H_test = H[:split], H[split:]
S_train, S_test = S[:split], S[split:]
R_train, R_test = R[:split], R[split:]
y_train, y_test = y[:split], y[split:]

# ----------------------------
# HYPERPARAMETERS
# ----------------------------
lr = 0.01
epochs = 1000
decay = 0.7

T_opt = 25
T_range = 15

# Learnable parameters
w_env = np.random.rand(4)     # environmental weights
w_couple = np.random.rand(3)  # interaction weights
```

```
alpha = 0.5   # adaptation rate
beta = 0.3    # threshold sensitivity

# ----------------------------
# FUNCTIONS
# ----------------------------
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def features(T, H, S, R):
    Ts = abs(T - T_opt) / T_range
    Hs = H / 100
    Ss = 1 - (S / 1023)

    base = np.array([Ts, Hs, Ss, R])

    interaction = np.array([
        Ts * Ss,
        Hs * Ss,
        Ts * Hs
    ])

    return base, interaction

# ----------------------------
# TRAINING
# ----------------------------
for epoch in range(epochs):

    memory = []
    resistance = 0.5  # plant adaptive resistance
    total_loss = 0

    for i in range(len(T_train)):

        base, inter = features(T_train[i], H_train[i], S_train[i], R_train[i])

        # Instant environmental stress
        instant = np.dot(w_env, base) + np.dot(w_couple, inter)

        # Memory field (energy accumulation)
        memory.append(instant)
        if len(memory) > 6:
            memory.pop(0)

        energy = sum(val * np.exp(-decay*j)
                     for j, val in enumerate(reversed(memory)))

        # Adaptive resistance (plant fights stress)
        resistance = resistance + alpha * (1 - energy) * 0.01
        resistance = np.clip(resistance, 0.1, 1.0)

        # Dynamic threshold (homeostasis)
        threshold = beta * resistance

        # Final stress probability
        output = energy - threshold
        pred = sigmoid(output)

        error = pred - y_train[i]
        total_loss += error**2

        # Update weights
        w_env -= lr * error * base
        w_couple -= lr * error * inter

    print(f"Epoch {epoch+1}, Loss: {total_loss:.4f}")

# ----------------------------
# EVALUATION
# ----------------------------
def evaluate(T, H, S, R, y):

    memory = []
    resistance = 0.5
    correct = 0

    for i in range(len(T)):

        base, inter = features(T[i], H[i], S[i], R[i])

        instant = np.dot(w_env, base) + np.dot(w_couple, inter)

        memory.append(instant)
        if len(memory) > 6:
            memory.pop(0)

        energy = sum(val * np.exp(-decay*j)
```

```
                    for j, val in enumerate(reversed(memory)))

        resistance = resistance + alpha * (1 - energy) * 0.01
        resistance = np.clip(resistance, 0.1, 1.0)

        threshold = beta * resistance

        pred = sigmoid(energy - threshold)
        label = 1 if pred > 0.5 else 0

        if label == y[i]:
            correct += 1

    return correct / len(T)

train_acc = evaluate(T_train, H_train, S_train, R_train, y_train)
test_acc = evaluate(T_test, H_test, S_test, R_test, y_test)

print("\n🌿 BASF-Net Results")
print("Train Accuracy:", train_acc)
print("Test Accuracy:", test_acc)
```

```
Epoch 1, Loss: 65.0056
Epoch 2, Loss: 58.0257
Epoch 3, Loss: 56.7931
Epoch 4, Loss: 56.5382
Epoch 5, Loss: 56.3360
Epoch 6, Loss: 56.1479
Epoch 7, Loss: 55.9694
Epoch 8, Loss: 55.7991
Epoch 9, Loss: 55.6360
Epoch 10, Loss: 55.4792
Epoch 11, Loss: 55.3281
Epoch 12, Loss: 55.1823
Epoch 13, Loss: 55.0413
Epoch 14, Loss: 54.9048
Epoch 15, Loss: 54.7724
Epoch 16, Loss: 54.6440
Epoch 17, Loss: 54.5193
Epoch 18, Loss: 54.3980
Epoch 19, Loss: 54.2801
Epoch 20, Loss: 54.1653
Epoch 21, Loss: 54.0535
Epoch 22, Loss: 53.9447
Epoch 23, Loss: 53.8385
Epoch 24, Loss: 53.7351
Epoch 25, Loss: 53.6342
Epoch 26, Loss: 53.5357
Epoch 27, Loss: 53.4395
Epoch 28, Loss: 53.3456
Epoch 29, Loss: 53.2539
Epoch 30, Loss: 53.1643
Epoch 31, Loss: 53.0766
Epoch 32, Loss: 52.9909
Epoch 33, Loss: 52.9070
Epoch 34, Loss: 52.8249
Epoch 35, Loss: 52.7445
Epoch 36, Loss: 52.6658
Epoch 37, Loss: 52.5886
Epoch 38, Loss: 52.5130
Epoch 39, Loss: 52.4389
Epoch 40, Loss: 52.3662
Epoch 41, Loss: 52.2949
Epoch 42, Loss: 52.2249
Epoch 43, Loss: 52.1562
Epoch 44, Loss: 52.0887
Epoch 45, Loss: 52.0224
Epoch 46, Loss: 51.9573
Epoch 47, Loss: 51.8933
Epoch 48, Loss: 51.8304
Epoch 49, Loss: 51.7685
Epoch 50, Loss: 51.7077
Epoch 51, Loss: 51.6479
Epoch 52, Loss: 51.5890
Epoch 53, Loss: 51.5310
Epoch 54, Loss: 51.4740
Epoch 55, Loss: 51.4178
Epoch 56, Loss: 51.3625
Epoch 57, Loss: 51.3080
Epoch 58, Loss: 51.2543
```

```
import numpy as np
import pandas as pd

np.random.seed(42)

rows = []

def generate_realistic_row():
    # Diverse environmental ranges
    T = np.random.uniform(10, 45)      # Temperature
    H = np.random.uniform(20, 100)     # Humidity
    S = np.random.uniform(200, 1000)   # Soil moisture
```

```
    R = np.random.choice([0, 1])       # Rain

    # Derived stress factors
    Ts = abs(T - 25) / 15
    Hs = H / 100
    Ss = 1 - (S / 1023)

    # Base stress logic (more realistic)
    stress_score = (
        0.4 * Ts +
        0.3 * Ss +
        0.2 * (1 - Hs) +
        0.1 * R
    )

    # Add nonlinear interactions
    stress_score += 0.2 * (Ts * Ss)
    stress_score += 0.1 * (Hs * Ss)

    # Add noise (important for realism)
    stress_score += np.random.normal(0, 0.05)

    # Convert to label
    stress = 1 if stress_score > 0.5 else 0

    return [round(T,2), round(H,2), round(S,2), R, stress]


# 🔥 Generate MANY rows
for _ in range(2500):
    rows.append(generate_realistic_row())

df = pd.DataFrame(rows, columns=[
    "Temperature", "Humidity", "Soil Moisture", "Rain", "Stress"
])

# ----------------------------
# BALANCE DATASET
# ----------------------------
majority = df[df["Stress"] == 1]
minority = df[df["Stress"] == 0]

if len(minority) < len(majority):
    minority = minority.sample(len(majority), replace=True, random_state=42)
else:
    majority = majority.sample(len(minority), replace=True, random_state=42)

df_balanced = pd.concat([majority, minority]).sample(frac=1, random_state=42)

# ----------------------------
# SAVE CSV FILE
# ----------------------------
df_balanced.to_csv("synthetic_plant_stress_data.csv", index=False)

print("✅ Dataset generated successfully!")
print("Total rows:", len(df_balanced))
print(df_balanced["Stress"].value_counts())
```

```
✅ Dataset generated successfully!
Total rows: 2968
Stress
0    1484
1    1484
Name: count, dtype: int64
```

```
real = pd.read_csv("Plant Stress Language decoder - Sheet1.csv")
synthetic = pd.read_csv("synthetic_plant_stress_data.csv")

final = pd.concat([real, synthetic])

# Shuffle (VERY IMPORTANT)
final = final.sample(frac=1, random_state=42)

# Save
# final.to_csv("final_combined_plant_stress_data.csv", index=False)
```

```
print(final["Stress"].value_counts())
```

```
Stress
1    1701
0    1583
Name: count, dtype: int64
```

```
final.to_csv("plant_stress_data.csv", index=False)
```

```
import numpy as np
import pandas as pd
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# ----------------------------
# LOAD DATA
# ----------------------------
data = pd.read_csv("plant_stress_data.csv")

# ----------------------------
# KEEP ONLY REQUIRED COLUMNS
# ----------------------------
required_cols = ["Temperature", "Humidity", "Soil Moisture", "Rain", "Stress"]
data = data[required_cols]

# ----------------------------
# FIX RAIN COLUMN
# ----------------------------
if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({
        "No Rain": 0,
        "Rain": 1
    })

# ----------------------------
# CONVERT TO NUMERIC
# ----------------------------
for col in required_cols:
    data[col] = pd.to_numeric(data[col], errors='coerce')

# Remove invalid rows
data = data.dropna()

# Shuffle dataset
data = data.sample(frac=1, random_state=42)

# ----------------------------
# EXTRACT FEATURES
# ----------------------------
T = data["Temperature"].values
H = data["Humidity"].values
S = data["Soil Moisture"].values
R = data["Rain"].values
y = data["Stress"].values

# ----------------------------
# BASF FEATURE FUNCTION (UNIQUE)
# ----------------------------
T_opt = 25
T_range = 15

def basf_features(T, H, S, R):

    # Ensure numeric
    T = float(T)
    H = float(H)
    S = float(S)
    R = float(R)

    # Normalized stress components
    Ts = abs(T - T_opt) / T_range
    Hs = H / 100
    Ss = 1 - (S / 1023)

    base = np.array([Ts, Hs, Ss, R], dtype=float)

    # Stress energy (environmental field)
    energy = np.dot(base, np.array([0.4, 0.3, 0.2, 0.1]))

    # Adaptive resistance (plant behavior)
    resistance = 0.5 + 0.1 * (1 - energy)

    # Dynamic threshold
    threshold = 0.3 * resistance

    return [energy, resistance, threshold]

# ----------------------------
# BUILD FEATURE DATASET
# ----------------------------
X = []

for i in range(len(T)):
    X.append(basf_features(T[i], H[i], S[i], R[i]))

X = np.array(X)

# ----------------------------
# TRAIN-TEST SPLIT
```

```
# ----------------------------
split = int(0.85 * len(X))

X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# ----------------------------
# MODEL (HYBRID FINAL MODEL)
# ----------------------------
model = RandomForestClassifier(
    n_estimators=300,
    max_depth=8,
    min_samples_split=5,
    random_state=42
)

model.fit(X_train, y_train)

# ----------------------------
# EVALUATION
# ----------------------------
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)

print("\n🔥 FINAL HYBRID MODEL RESULTS")
print("Train Accuracy:", accuracy_score(y_train, train_pred))
print("Test Accuracy:", accuracy_score(y_test, test_pred))

print("\n📊 Classification Report:")
print(classification_report(y_test, test_pred))
```

```
🔥 FINAL HYBRID MODEL RESULTS
Train Accuracy: 0.8582089552238806
Test Accuracy: 0.7708333333333334

📊 Classification Report:
              precision    recall  f1-score   support

           0       0.64      0.50      0.56        14
           1       0.81      0.88      0.85        34

    accuracy                           0.77        48
   macro avg       0.72      0.69      0.70        48
weighted avg       0.76      0.77      0.76        48
```

```
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.model_selection import train_test_split

# ----------------------------
# LOAD DATA
# ----------------------------
data = pd.read_csv("plant_stress_data.csv")

# ----------------------------
# KEEP ONLY REQUIRED COLUMNS
# ----------------------------
required_cols = ["Temperature", "Humidity", "Soil Moisture", "Rain", "Stress"]
data = data[required_cols]

# ----------------------------
# FIX RAIN COLUMN
# ----------------------------
if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({
        "No Rain": 0,
        "Rain": 1
    })

# ----------------------------
# CONVERT TO NUMERIC
# ----------------------------
for col in required_cols:
    data[col] = pd.to_numeric(data[col], errors='coerce')

# Drop invalid rows
data = data.dropna()

# Shuffle dataset
data = data.sample(frac=1, random_state=42)

# ----------------------------
# EXTRACT FEATURES
# ----------------------------
T = data["Temperature"].values
```

```
H = data["Humidity"].values
S = data["Soil Moisture"].values
R = data["Rain"].values
y = data["Stress"].values

# ----------------------------
# ENHANCED BASF FEATURE FUNCTION
# ----------------------------
T_opt = 25
T_range = 15

def basf_features(T, H, S, R):

    T = float(T)
    H = float(H)
    S = float(S)
    R = float(R)

    Ts = abs(T - T_opt) / T_range
    Hs = H / 100
    Ss = 1 - (S / 1023)

    # NEW: variability feature
    variability = np.std([Ts, Hs, Ss])

    # Enhanced stress energy
    energy = (
        0.4 * Ts +
        0.3 * Ss +
        0.2 * (1 - Hs) +
        0.1 * R +
        0.2 * Ts * Ss +
        0.1 * Hs * Ss
    )

    # Adaptive resistance
    resistance = 0.5 + 0.1 * (1 - energy)

    # Dynamic threshold
    threshold = 0.3 * resistance

    return [energy, resistance, threshold, variability]

# ----------------------------
# BUILD FEATURE DATASET
# ----------------------------
X = []

for i in range(len(T)):
    X.append(basf_features(T[i], H[i], S[i], R[i]))

X = np.array(X)

# ----------------------------
# STRATIFIED TRAIN-TEST SPLIT
# ----------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.15,
    random_state=42,
    stratify=y   # 🔥 fixes imbalance
)

# ----------------------------
# MODEL (UPGRADED)
# ----------------------------
model = RandomForestClassifier(
    n_estimators=400,
    max_depth=10,
    min_samples_split=5,
    class_weight="balanced",  # 🔥 improves class 0
    random_state=42
)

model.fit(X_train, y_train)

# ----------------------------
# EVALUATION
# ----------------------------
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)

print("\n🔥 FINAL OPTIMIZED MODEL RESULTS")
print("Train Accuracy:", accuracy_score(y_train, train_pred))
print("Test Accuracy:", accuracy_score(y_test, test_pred))

print("\n📊 Classification Report:")
print(classification_report(y_test, test_pred))
```

```
🔥 FINAL OPTIMIZED MODEL RESULTS
Train Accuracy: 0.9328358208955224
Test Accuracy: 0.7083333333333334

📊 Classification Report:
              precision    recall  f1-score   support

           0       0.53      0.67      0.59        15
           1       0.83      0.73      0.77        33

    accuracy                           0.71        48
   macro avg       0.68      0.70      0.68        48
weighted avg       0.73      0.71      0.72        48
```

```
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.model_selection import train_test_split

# ----------------------------
# LOAD DATA
# ----------------------------
data = pd.read_csv("plant_stress_data.csv")

# ----------------------------
# KEEP ONLY REQUIRED COLUMNS
# ----------------------------
required_cols = ["Temperature", "Humidity", "Soil Moisture", "Rain", "Stress"]
data = data[required_cols]

# ----------------------------
# FIX RAIN COLUMN
# ----------------------------
if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({
        "No Rain": 0,
        "Rain": 1
    })

# ----------------------------
# CONVERT TO NUMERIC
# ----------------------------
for col in required_cols:
    data[col] = pd.to_numeric(data[col], errors='coerce')

# Remove invalid rows
data = data.dropna()

# Shuffle dataset
data = data.sample(frac=1, random_state=42)

# ----------------------------
# EXTRACT VALUES
# ----------------------------
T = data["Temperature"].values
H = data["Humidity"].values
S = data["Soil Moisture"].values
R = data["Rain"].values
y = data["Stress"].values

# ----------------------------
# SIMPLIFIED BASF FEATURES (REDUCES OVERFITTING)
# ----------------------------
T_opt = 25
T_range = 15

def basf_features(T, H, S, R):

    T = float(T)
    H = float(H)
    S = float(S)
    R = float(R)

    # Normalize
    Ts = abs(T - T_opt) / T_range
    Hs = H / 100
    Ss = 1 - (S / 1023)

    # Core stress energy (simplified)
    energy = (
        0.5 * Ts +
        0.3 * Ss +
        0.2 * (1 - Hs)
    )
```

```
    # Variability feature
    variability = np.std([Ts, Hs, Ss])

    # Adaptive resistance
    resistance = 0.5 + 0.1 * (1 - energy)

    return [energy, variability, resistance]

# ----------------------------
# BUILD FEATURE DATASET
# ----------------------------
X = []

for i in range(len(T)):
    X.append(basf_features(T[i], H[i], S[i], R[i]))

X = np.array(X)

# ----------------------------
# STRATIFIED TRAIN-TEST SPLIT
# ----------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.15,
    random_state=42,
    stratify=y
)

# ----------------------------
# FINAL MODEL (OVERFITTING FIXED)
# ----------------------------
model = RandomForestClassifier(
    n_estimators=200,
    max_depth=6,             # 🔥 prevents overfitting
    min_samples_split=10,    # smoother learning
    min_samples_leaf=5,      # avoids noise learning
    class_weight="balanced",
    random_state=42
)

model.fit(X_train, y_train)

# ----------------------------
# EVALUATION
# ----------------------------
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)

print("\n🔥 FINAL STABLE MODEL RESULTS")
print("Train Accuracy:", accuracy_score(y_train, train_pred))
print("Test Accuracy:", accuracy_score(y_test, test_pred))

print("\n📊 Classification Report:")
print(classification_report(y_test, test_pred))
```

```
🔥 FINAL STABLE MODEL RESULTS
Train Accuracy: 0.8171641791044776
Test Accuracy: 0.75

📊 Classification Report:
              precision    recall  f1-score   support

           0       0.56      0.93      0.70        15
           1       0.96      0.67      0.79        33

    accuracy                           0.75        48
   macro avg       0.76      0.80      0.74        48
weighted avg       0.83      0.75      0.76        48
```

```
import numpy as np
import pandas as pd
from sklearn.metrics import accuracy_score, classification_report
from sklearn.model_selection import train_test_split

# ----------------------------
# LOAD DATA
# ----------------------------
data = pd.read_csv("plant_stress_data.csv")

# Keep only required columns
data = data[["Temperature", "Humidity", "Soil Moisture", "Rain", "Stress"]]

# Fix Rain column
if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({"No Rain": 0, "Rain": 1})

# Convert to numeric
```

```
for col in data.columns:
    data[col] = pd.to_numeric(data[col], errors='coerce')

data = data.dropna()
data = data.sample(frac=1, random_state=42)

# Extract
T = data["Temperature"].values
H = data["Humidity"].values
S = data["Soil Moisture"].values
R = data["Rain"].values
y = data["Stress"].values

# ----------------------------
# SPLIT (STRATIFIED)
# ----------------------------
X_dummy = np.zeros(len(y))  # placeholder

X_train_idx, X_test_idx, y_train, y_test = train_test_split(
    np.arange(len(y)), y,
    test_size=0.15,
    random_state=42,
    stratify=y
)

# ----------------------------
# BASF ENERGY FUNCTION
# ----------------------------
def basf_energy(T, H, S, R):

    Ts = abs(T - 25) / 15
    Hs = H / 100
    Ss = 1 - (S / 1023)

    energy = (
        0.5 * Ts +
        0.3 * Ss +
        0.2 * (1 - Hs)
    )

    return energy

# ----------------------------
# DYNAMIC MEMORY MODEL
# ----------------------------
def run_model(indices):

    memory = []
    decay = 0.6
    alpha = 0.05   # adaptation rate
    threshold = 0.5

    preds = []

    for i in indices:

        e = basf_energy(T[i], H[i], S[i], R[i])

        # Store memory
        memory.append(e)
        if len(memory) > 5:
            memory.pop(0)

        # Temporal accumulation
        cumulative = 0
        for j, val in enumerate(reversed(memory)):
            cumulative += val * np.exp(-decay * j)

        # Adaptive threshold
        threshold = threshold + alpha * (0.5 - cumulative)
        threshold = np.clip(threshold, 0.3, 0.7)

        # Final prediction
        pred = 1 if cumulative > threshold else 0
        preds.append(pred)

    return np.array(preds)

# ----------------------------
# TRAIN & TEST
# ----------------------------
train_pred = run_model(X_train_idx)
test_pred = run_model(X_test_idx)

print("\n🔥 BASF-DMNF MODEL RESULTS")
print("Train Accuracy:", accuracy_score(y_train, train_pred))
print("Test Accuracy:", accuracy_score(y_test, test_pred))
```

```
print("\n📊 Classification Report:")
print(classification_report(y_test, test_pred))
```

```
🔥 BASF-DMNF MODEL RESULTS
Train Accuracy: 0.6828358208955224
Test Accuracy: 0.6666666666666666

📊 Classification Report:
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        15
           1       0.68      0.97      0.80        33

    accuracy                           0.67        48
   macro avg       0.34      0.48      0.40        48
weighted avg       0.47      0.67      0.55        48
```

```
import numpy as np
import pandas as pd
from sklearn.metrics import accuracy_score, classification_report
from sklearn.model_selection import train_test_split

# ----------------------------
# LOAD DATA
# ----------------------------
data = pd.read_csv("plant_stress_data.csv")

data = data[["Temperature", "Humidity", "Soil Moisture", "Rain", "Stress"]]

if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({"No Rain": 0, "Rain": 1})

for col in data.columns:
    data[col] = pd.to_numeric(data[col], errors="coerce")

data = data.dropna()
data = data.sample(frac=1, random_state=42)

T = data["Temperature"].values
H = data["Humidity"].values
S = data["Soil Moisture"].values
R = data["Rain"].values
y = data["Stress"].values

# ----------------------------
# FEATURE FUNCTION
# ----------------------------
def features(i):
    return np.array([
        abs(T[i] - 25) / 15,
        H[i] / 100,
        1 - (S[i] / 1023),
        R[i]
    ])

# ----------------------------
# GENETIC WEIGHTS (KEY IDEA)
# ----------------------------
np.random.seed(42)
weights = np.random.uniform(-1, 1, 4)

lr = 0.01

# ----------------------------
# TRAINING (EVOLUTION STYLE)
# ----------------------------
for epoch in range(50):

    for i in range(len(T)):

        x = features(i)

        # prediction
        score = np.dot(weights, x)
        pred = 1 if score > 0 else 0

        error = y[i] - pred

        # evolutionary update
        weights += lr * error * x

# ----------------------------
# PREDICTION
# ----------------------------
def predict(i):
    x = features(i)
    score = np.dot(weights, x)
```

```
    return 1 if score > 0 else 0

# ----------------------------
# SPLIT
# ----------------------------
idx = np.arange(len(y))

train_idx, test_idx = train_test_split(
    idx,
    test_size=0.15,
    stratify=y,
    random_state=42
)

train_pred = [predict(i) for i in train_idx]
test_pred = [predict(i) for i in test_idx]

print("\n🌿 BASF-GA MODEL RESULTS")
print("Train Accuracy:", accuracy_score(y[train_idx], train_pred))
print("Test Accuracy:", accuracy_score(y[test_idx], test_pred))

print("\n📊 Classification Report:")
print(classification_report(y[test_idx], test_pred))
```

```
🌿 BASF-GA MODEL RESULTS
Train Accuracy: 0.746268656716418
Test Accuracy: 0.7083333333333334

📊 Classification Report:
              precision    recall  f1-score   support

           0       0.67      0.13      0.22        15
           1       0.71      0.97      0.82        33

    accuracy                           0.71        48
   macro avg       0.69      0.55      0.52        48
weighted avg       0.70      0.71      0.63        48
```

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# ----------------------------
# LOAD DATA
# ----------------------------
data = pd.read_csv("plant_stress_data.csv")

data = data[["Temperature", "Humidity", "Soil Moisture", "Rain", "Stress"]]

if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({"No Rain": 0, "Rain": 1})

for col in data.columns:
    data[col] = pd.to_numeric(data[col], errors="coerce")

data = data.dropna()
data = data.sample(frac=1, random_state=42)

T = data["Temperature"].values
H = data["Humidity"].values
S = data["Soil Moisture"].values
R = data["Rain"].values
y = data["Stress"].values

# ----------------------------
# 1️⃣ SENSORY LAYER
# ----------------------------
def sense(i):

    heat = abs(T[i] - 25) / 15
    dry = 1 - (S[i] / 1023)
    humid = 1 - (H[i] / 100)

    return np.array([heat, dry, humid, R[i]])

# ----------------------------
# 2️⃣ ADAPTIVE WEIGHT SYSTEM
# ----------------------------
weights = np.array([0.4, 0.3, 0.2, 0.1])

learning_rate = 0.02

def adapt_weights(x, error):

    global weights
    weights = weights + learning_rate * error * x
```

```
    weights = np.clip(weights, -1, 1)

# ----------------------------
# 3️⃣ MODEL TRAINING (BIOLOGICAL LEARNING LOOP)
# ----------------------------
epochs = 10

for epoch in range(epochs):

    for i in range(len(T)):

        x = sense(i)

        score = np.dot(weights, x)
        prob = 1 / (1 + np.exp(-score))

        pred = 1 if prob > 0.5 else 0

        error = y[i] - pred

        adapt_weights(x, error)

# ----------------------------
# 4️⃣ PREDICTION FUNCTION
# ----------------------------
def predict(i):

    x = sense(i)
    score = np.dot(weights, x)
    prob = 1 / (1 + np.exp(-score))

    return 1 if prob > 0.5 else 0

# ----------------------------
# 5️⃣ TRAIN / TEST SPLIT
# ----------------------------
idx = np.arange(len(y))

train_idx, test_idx = train_test_split(
    idx,
    test_size=0.15,
    stratify=y,
    random_state=42
)

train_pred = [predict(i) for i in train_idx]
test_pred = [predict(i) for i in test_idx]

print("\n🌿 D.A.R.W.I.N MODEL RESULTS")
print("Train Accuracy:", accuracy_score(y[train_idx], train_pred))
print("Test Accuracy:", accuracy_score(y[test_idx], test_pred))

print("\n📊 Classification Report:")
print(classification_report(y[test_idx], test_pred))
```

```
🌿 D.A.R.W.I.N MODEL RESULTS
Train Accuracy: 0.6828358208955224
Test Accuracy: 0.6458333333333334

📊 Classification Report:
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        15
           1       0.67      0.94      0.78        33

    accuracy                           0.65        48
   macro avg       0.34      0.47      0.39        48
weighted avg       0.46      0.65      0.54        48
```

```
import numpy as np
import pandas as pd
from sklearn.metrics import accuracy_score

# ----------------------------
# LOAD + CLEAN DATA
# ----------------------------
data = pd.read_csv("plant_stress_data.csv")

if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({"No Rain": 0, "Rain": 1})

cols = ["Temperature", "Humidity", "Soil Moisture", "Rain", "Stress"]
for col in cols:
    data[col] = pd.to_numeric(data[col], errors='coerce')

data = data.dropna()
```

```
# ----------------------------
# NORMALIZATION (IMPORTANT FIX)
# ----------------------------
data["Temperature"] = (data["Temperature"] - data["Temperature"].mean()) / data["Temperature"].std()
data["Humidity"] = (data["Humidity"] - data["Humidity"].mean()) / data["Humidity"].std()
data["Soil Moisture"] = (data["Soil Moisture"] - data["Soil Moisture"].mean()) / data["Soil Moisture"].std()

# ----------------------------
# FEATURES
# ----------------------------
T = data["Temperature"].values
H = data["Humidity"].values
S = data["Soil Moisture"].values
R = data["Rain"].values
y = data["Stress"].values

# ----------------------------
# TRAIN-TEST SPLIT (FIXED RANDOM SHUFFLE)
# ----------------------------
indices = np.arange(len(T))
np.random.shuffle(indices)

split = int(0.85 * len(T))
train_idx, test_idx = indices[:split], indices[split:]

T_train, H_train, S_train, R_train, y_train = T[train_idx], H[train_idx], S[train_idx], R[train_idx], y[train_idx]
T_test, H_test, S_test, R_test, y_test = T[test_idx], H[test_idx], S[test_idx], R[test_idx], y[test_idx]

# ----------------------------
# HYPERPARAMETERS (STABILIZED)
# ----------------------------
lr = 0.005
epochs = 400
decay = 0.5
momentum = 0.85

T_opt = 0   # normalized data → center is 0
T_range = 1

# ----------------------------
# WEIGHTS
# ----------------------------
w_main = np.random.randn(4) * 0.1
w_inter = np.random.randn(3) * 0.1
w_gate = np.random.randn(4) * 0.1

v_main = np.zeros(4)
v_inter = np.zeros(3)
v_gate = np.zeros(4)

# ----------------------------
# ACTIVATION
# ----------------------------
def sigmoid(x):
    x = np.clip(x, -10, 10)   # FIX: prevents saturation
    return 1 / (1 + np.exp(-x))

# ----------------------------
# FEATURE ENGINEERING
# ----------------------------
def features(T, H, S, R):

    Ts = abs(T - T_opt) / T_range
    Hs = H
    Ss = S
    Rs = R

    base = np.array([Ts, Hs, Ss, Rs])

    inter = np.array([
        Ts * Ss,
        Hs * Ss,
        Ts * Hs
    ])

    return base, inter

# ----------------------------
# TRAINING
# ----------------------------
for epoch in range(epochs):

    memory = []

    lr_epoch = lr * (0.995 ** epoch)  # FIX: learning rate decay

    total_loss = 0
```

```
    for i in range(len(T_train)):

        base, inter = features(T_train[i], H_train[i], S_train[i], R_train[i])

        instant = np.dot(w_main, base) + np.dot(w_inter, inter)

        memory.append(instant)
        if len(memory) > 5:
            memory.pop(0)

        # FIX: stable memory (no exponential explosion)
        mem_effect = np.mean(memory)

        gate = sigmoid(np.dot(w_gate, base))

        combined = gate * mem_effect + (1 - gate) * instant
        combined = np.clip(combined, -10, 10)

        pred = sigmoid(combined)

        error = pred - y_train[i]
        total_loss += error ** 2

        # ----------------------------
        # GRADIENT UPDATE (STABLE)
        # ----------------------------
        grad_main = error * base
        grad_inter = error * inter
        grad_gate = error * base

        v_main = momentum * v_main + lr_epoch * grad_main
        v_inter = momentum * v_inter + lr_epoch * grad_inter
        v_gate = momentum * v_gate + lr_epoch * grad_gate

        w_main -= v_main
        w_inter -= v_inter
        w_gate -= v_gate

    if epoch % 50 == 0:
        print(f"Epoch {epoch}, Loss: {total_loss:.4f}")

# ----------------------------
# EVALUATION FUNCTION
# ----------------------------
def evaluate(T, H, S, R, y):

    memory = []
    preds = []

    for i in range(len(T)):

        base, inter = features(T[i], H[i], S[i], R[i])

        instant = np.dot(w_main, base) + np.dot(w_inter, inter)

        memory.append(instant)
        if len(memory) > 5:
            memory.pop(0)

        mem_effect = np.mean(memory)

        gate = sigmoid(np.dot(w_gate, base))

        combined = gate * mem_effect + (1 - gate) * instant
        combined = np.clip(combined, -10, 10)

        pred = sigmoid(combined)

        preds.append(pred)

    preds = np.array(preds)

    # FIX: dynamic threshold (better generalization)
    threshold = np.mean(preds)

    final = (preds > threshold).astype(int)

    return accuracy_score(y, final)

# ----------------------------
# RESULTS
# ----------------------------
train_acc = evaluate(T_train, H_train, S_train, R_train, y_train)
test_acc = evaluate(T_test, H_test, S_test, R_test, y_test)

print("\n🔥 FINAL IMPROVED RESULTS")
print("Train Accuracy:", train_acc)
print("Test Accuracy:", test_acc)
```

```
Epoch 0, Loss: 54.9842
Epoch 50, Loss: 51.1846
Epoch 100, Loss: 51.0225
Epoch 150, Loss: 50.8913
Epoch 200, Loss: 50.7843
Epoch 250, Loss: 50.6981
Epoch 300, Loss: 50.6294
Epoch 350, Loss: 50.5750

🔥 FINAL IMPROVED RESULTS
Train Accuracy: 0.7089552238805971
Test Accuracy: 0.625
```

```
import numpy as np
import pandas as pd
from sklearn.metrics import accuracy_score

# ----------------------------
# LOAD DATA
# ----------------------------
data = pd.read_csv("plant_stress_data.csv")

if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({"No Rain": 0, "Rain": 1})

cols = ["Temperature", "Humidity", "Soil Moisture", "Rain", "Stress"]
for col in cols:
    data[col] = pd.to_numeric(data[col], errors='coerce')

data = data.dropna()

# ----------------------------
# NORMALIZATION
# ----------------------------
for col in ["Temperature", "Humidity", "Soil Moisture"]:
    data[col] = (data[col] - data[col].mean()) / data[col].std()

T = data["Temperature"].values
H = data["Humidity"].values
S = data["Soil Moisture"].values
R = data["Rain"].values
y = data["Stress"].values

# ----------------------------
# TRAIN TEST SPLIT
# ----------------------------
idx = np.arange(len(T))
np.random.shuffle(idx)

split = int(0.85 * len(T))
train_idx, test_idx = idx[:split], idx[split:]

# ----------------------------
# MODEL PARAMETERS
# ----------------------------
w = np.random.randn(5) * 0.1  # main weights
lr = 0.01
momentum = 0.85

velocity = np.zeros(5)

T_decay = 0.85  # temporal decay factor

# ----------------------------
# FEATURE ENGINEERING
# ----------------------------
def features(i):

    heat = abs(T[i])
    humidity = H[i]
    soil = S[i]
    rain = R[i]

    interaction = heat * soil

    return np.array([heat, humidity, soil, rain, interaction])

# ----------------------------
# ATTENTION MEMORY SYSTEM
# ----------------------------
memory = []

def attention_memory(new_value):

    memory.append(new_value)

    if len(memory) > 7:
        memory.pop(0)
```

```
    weights = np.array([T_decay ** i for i in range(len(memory))])
    weights = weights[::-1]

    return np.sum(np.array(memory) * weights) / (np.sum(weights) + 1e-6)

# ----------------------------
# SIGMOID
# ----------------------------
def sigmoid(x):
    x = np.clip(x, -10, 10)
    return 1 / (1 + np.exp(-x))

# ----------------------------
# TRAINING
# ----------------------------
for epoch in range(250):

    lr_epoch = lr * (0.99 ** epoch)

    memory.clear()

    for i in train_idx:

        x = features(i)

        raw = np.dot(w, x)

        # temporal memory
        mem = attention_memory(raw)

        # combine current + memory
        combined = 0.6 * raw + 0.4 * mem

        pred = sigmoid(combined)

        error = pred - y[i]

        # gradient scaling (IMPORTANT)
        grad = error * x
        grad = np.clip(grad, -1, 1)

        velocity = momentum * velocity + lr_epoch * grad
        w -= velocity

# ----------------------------
# CALIBRATED THRESHOLD
# ----------------------------
train_scores = []

memory.clear()

for i in train_idx:
    x = features(i)
    raw = np.dot(w, x)
    mem = attention_memory(raw)
    score = sigmoid(0.6 * raw + 0.4 * mem)
    train_scores.append(score)

threshold = np.mean(train_scores)

# ----------------------------
# EVALUATION FUNCTION
# ----------------------------
def evaluate(indices):

    memory.clear()
    preds = []

    for i in indices:

        x = features(i)
        raw = np.dot(w, x)
        mem = attention_memory(raw)

        score = sigmoid(0.6 * raw + 0.4 * mem)

        preds.append(score)

    preds = np.array(preds)
    final = (preds > threshold).astype(int)

    return final

# ----------------------------
# RESULTS
# ----------------------------
train_pred = evaluate(train_idx)
```

```
test_pred = evaluate(test_idx)

print("\n🌿 S.T.G.S.N MODEL RESULTS")
print("Train Accuracy:", accuracy_score(y[train_idx], train_pred))
print("Test Accuracy:", accuracy_score(y[test_idx], test_pred))
```

```
🌿 S.T.G.S.N MODEL RESULTS
Train Accuracy: 0.7238805970149254
Test Accuracy: 0.7708333333333334
```

```
import numpy as np
import pandas as pd
from sklearn.metrics import accuracy_score

# ----------------------------
# LOAD DATA
# ----------------------------
data = pd.read_csv("plant_stress_data.csv")

if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({"No Rain": 0, "Rain": 1})

cols = ["Temperature", "Humidity", "Soil Moisture", "Rain", "Stress"]
for col in cols:
    data[col] = pd.to_numeric(data[col], errors="coerce")

data = data.dropna()

# ----------------------------
# NORMALIZATION (IMPORTANT UPGRADE)
# ----------------------------
for col in ["Temperature", "Humidity", "Soil Moisture"]:
    data[col] = (data[col] - data[col].mean()) / data[col].std()

T = data["Temperature"].values
H = data["Humidity"].values
S = data["Soil Moisture"].values
R = data["Rain"].values
y = data["Stress"].values

# ----------------------------
# TRAIN-TEST SPLIT
# ----------------------------
idx = np.arange(len(T))
np.random.shuffle(idx)

split = int(0.85 * len(T))
train_idx, test_idx = idx[:split], idx[split:]

# ----------------------------
# MODEL PARAMETERS
# ----------------------------
w = np.random.randn(9) * 0.05
lr = 0.008
momentum = 0.88
velocity = np.zeros(9)

memory = []
decay = 0.85

# ----------------------------
# FEATURE ENGINEERING (UPGRADED)
# ----------------------------
def features(i):

    heat = abs(T[i])
    humidity = H[i]
    soil = S[i]
    rain = R[i]

    # nonlinear expansion (BIG UPGRADE)
    return np.array([
        heat,
        humidity,
        soil,
        rain,
        heat * soil,
        humidity * soil,
        heat * humidity,
        soil ** 2,
        heat ** 2
    ])

# ----------------------------
# TEMPORAL MEMORY (IMPROVED)
# ----------------------------
def memory_effect(value):
```

```
    memory.append(value)

    if len(memory) > 8:
        memory.pop(0)

    weights = np.array([decay ** i for i in range(len(memory))])[::-1]

    return np.sum(np.array(memory) * weights) / (np.sum(weights) + 1e-6)

# ----------------------------
# SIGMOID
# ----------------------------
def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -8, 8)))

# ----------------------------
# TRAINING LOOP
# ----------------------------
for epoch in range(180):

    lr_epoch = lr * (0.995 ** epoch)
    memory.clear()

    for i in train_idx:

        x = features(i)

        raw = np.dot(w, x)

        mem = memory_effect(raw)

        combined = 0.7 * raw + 0.3 * mem

        pred = sigmoid(combined)

        error = pred - y[i]

        # improved gradient (less clipping)
        grad = error * x

        velocity = momentum * velocity + lr_epoch * grad
        w -= velocity

# ----------------------------
# THRESHOLD LEARNING
# ----------------------------
train_scores = []

memory.clear()

for i in train_idx:
    x = features(i)
    raw = np.dot(w, x)
    mem = memory_effect(raw)
    score = sigmoid(0.7 * raw + 0.3 * mem)
    train_scores.append(score)

threshold = np.median(train_scores)  # better than mean

# ----------------------------
# EVALUATION
# ----------------------------
def evaluate(indices):

    memory.clear()
    preds = []

    for i in indices:

        x = features(i)

        raw = np.dot(w, x)
        mem = memory_effect(raw)

        score = sigmoid(0.7 * raw + 0.3 * mem)

        preds.append(score)

    preds = np.array(preds)

    return (preds > threshold).astype(int)

# ----------------------------
# RESULTS
# ----------------------------
train_pred = evaluate(train_idx)
test_pred = evaluate(test_idx)
```

```
print("\n🌿 STGSN++ MODEL RESULTS")
print("Train Accuracy:", accuracy_score(y[train_idx], train_pred))
print("Test Accuracy:", accuracy_score(y[test_idx], test_pred))
```

```
🌿 STGSN++ MODEL RESULTS
Train Accuracy: 0.7835820895522388
Test Accuracy: 0.7708333333333334
```

```
import numpy as np
import pandas as pd
from sklearn.metrics import accuracy_score, classification_report, precision_score, recall_score, f1_score

# ----------------------------
# LOAD DATA
# ----------------------------
data = pd.read_csv("plant_stress_data.csv")

if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({"No Rain": 0, "Rain": 1})

cols = ["Temperature", "Humidity", "Soil Moisture", "Rain", "Stress"]
for col in cols:
    data[col] = pd.to_numeric(data[col], errors="coerce")

data = data.dropna()

# ----------------------------
# NORMALIZATION
# ----------------------------
for col in ["Temperature", "Humidity", "Soil Moisture"]:
    data[col] = (data[col] - data[col].mean()) / data[col].std()

T = data["Temperature"].values
H = data["Humidity"].values
S = data["Soil Moisture"].values
R = data["Rain"].values
y = data["Stress"].values

# ----------------------------
# SPLIT
# ----------------------------
idx = np.arange(len(T))
np.random.shuffle(idx)

split = int(0.85 * len(T))
train_idx, test_idx = idx[:split], idx[split:]

# ----------------------------
# MODEL
# ----------------------------
w = np.random.randn(9) * 0.05
lr = 0.008
momentum = 0.88
velocity = np.zeros(9)

memory = []
decay = 0.85

# ----------------------------
# FEATURES
# ----------------------------
def features(i):

    heat = abs(T[i])
    humidity = H[i]
    soil = S[i]
    rain = R[i]

    return np.array([
        heat,
        humidity,
        soil,
        rain,
        1.2 * heat * soil,
        1.2 * humidity * soil,
        1.1 * heat * humidity,
        soil ** 2,
        heat ** 2
    ])

# ----------------------------
# MEMORY (IMPROVED)
# ----------------------------
def memory_effect():

    if len(memory) == 0:
```

```
        return 0

    weights = np.exp(-decay * np.arange(len(memory)))[::-1]
    return np.sum(np.array(memory) * weights) / (np.sum(weights) + 1e-6)

# ----------------------------
# SIGMOID
# ----------------------------
def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -8, 8)))

# ----------------------------
# TRAINING
# ----------------------------
for epoch in range(180):

    lr_epoch = lr * (0.995 ** epoch)
    memory.clear()

    for i in train_idx:

        x = features(i)

        raw = np.dot(w, x)

        memory.append(raw)
        if len(memory) > 8:
            memory.pop(0)

        mem = memory_effect()

        combined = 0.7 * raw + 0.3 * mem

        pred = sigmoid(combined)

        error = pred - y[i]

        grad = error * x

        velocity = momentum * velocity + lr_epoch * grad
        w -= velocity

# ----------------------------
# PREDICTIONS
# ----------------------------
def predict(indices):

    memory.clear()
    scores = []

    for i in indices:

        x = features(i)
        raw = np.dot(w, x)

        memory.append(raw)
        if len(memory) > 8:
            memory.pop(0)

        mem = memory_effect()

        score = sigmoid(0.7 * raw + 0.3 * mem)

        scores.append(score)

    scores = np.array(scores)

    threshold = np.percentile(scores, 55)

    return (scores > threshold).astype(int)

# ----------------------------
# RESULTS
# ----------------------------
train_pred = predict(train_idx)
test_pred = predict(test_idx)

print("\n🌿 FINAL STGSN++ EVALUATION")

print("\n📊 ACCURACY")
print("Train Accuracy:", accuracy_score(y[train_idx], train_pred))
print("Test Accuracy:", accuracy_score(y[test_idx], test_pred))

print("\n📊 PRECISION (Stress=1)")
print("Train Precision:", precision_score(y[train_idx], train_pred))
print("Test Precision:", precision_score(y[test_idx], test_pred))

print("\n📊 RECALL (Stress=1)")
```

```
print("Test Recall:", recall_score(y[test_idx], test_pred))

print("\n📊 F1 SCORE")
print("Test F1:", f1_score(y[test_idx], test_pred))

print("\n📊 FULL REPORT")
print(classification_report(y[test_idx], test_pred))
```

```
🌿 FINAL STGSN++ EVALUATION

📊 ACCURACY
Train Accuracy: 0.7350746268656716
Test Accuracy: 0.7291666666666666

📊 PRECISION (Stress=1)
Train Precision: 0.9752066115702479
Test Precision: 0.9090909090909091

📊 RECALL (Stress=1)
Test Recall: 0.6451612903225806

📊 F1 SCORE
Test F1: 0.7547169811320755

📊 FULL REPORT
              precision    recall  f1-score   support

           0       0.58      0.88      0.70        17
           1       0.91      0.65      0.75        31

    accuracy                           0.73        48
   macro avg       0.74      0.76      0.73        48
weighted avg       0.79      0.73      0.73        48
```

```
import numpy as np
import pandas as pd
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# ----------------------------
# LOAD DATA
# ----------------------------
data = pd.read_csv("plant_stress_data.csv")

if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({"No Rain": 0, "Rain": 1})

cols = ["Temperature", "Humidity", "Soil Moisture", "Rain", "Stress"]
for col in cols:
    data[col] = pd.to_numeric(data[col], errors="coerce")

data = data.dropna()

# ----------------------------
# NORMALIZATION
# ----------------------------
for col in ["Temperature", "Humidity", "Soil Moisture"]:
    data[col] = (data[col] - data[col].mean()) / data[col].std()

T = data["Temperature"].values
H = data["Humidity"].values
S = data["Soil Moisture"].values
R = data["Rain"].values
y = data["Stress"].values

# ----------------------------
# TRAIN TEST SPLIT
# ----------------------------
idx = np.arange(len(T))
np.random.shuffle(idx)

split = int(0.85 * len(T))
train_idx, test_idx = idx[:split], idx[split:]

# ----------------------------
# MODEL PARAMETERS
# ----------------------------
w = np.random.randn(6) * 0.1
lr = 0.01
momentum = 0.9
velocity = np.zeros(6)

# ----------------------------
# FEATURE ENGINEERING (RISK BASED)
# ----------------------------
def features(i):

    heat = abs(T[i])
    humidity = H[i]
```

```
    soil = S[i]
    rain = R[i]

    # 🌿 risk signals
    drought_risk = heat * soil
    humidity_risk = -humidity
    stress_pressure = heat + soil - humidity

    return np.array([
        heat,
        soil,
        rain,
        drought_risk,
        humidity_risk,
        stress_pressure
    ])

# ----------------------------
# SIGMOID
# ----------------------------
def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -10, 10)))

# ----------------------------
# TRAINING (ASYMMETRIC LEARNING)
# ----------------------------
for epoch in range(200):

    lr_epoch = lr * (0.995 ** epoch)

    for i in train_idx:

        x = features(i)

        raw = np.dot(w, x)

        prob = sigmoid(raw)

        # ⚠️ ASYMMETRIC LOSS (IMPORTANT)
        # penalize missed stress MORE than false alarms
        if y[i] == 1:
            error = (prob - y[i]) * 2.2   # boost stress learning
        else:
            error = (prob - y[i]) * 0.8   # reduce penalty for false alarms

        grad = error * x

        velocity = 0.9 * velocity + lr_epoch * grad
        w -= velocity

# ----------------------------
# PREDICTION (LOWER THRESHOLD FOR RECALL)
# ----------------------------
def predict(indices):

    preds = []

    for i in indices:

        x = features(i)
        raw = np.dot(w, x)
        prob = sigmoid(raw)

        # 🌿 LOWER threshold → improves recall
        preds.append(1 if prob > 0.42 else 0)

    return np.array(preds)

# ----------------------------
# RESULTS
# ----------------------------
train_pred = predict(train_idx)
test_pred = predict(test_idx)

print("\n🌿 E.W.S.D MODEL RESULTS (EARLY WARNING SYSTEM)")

print("\n📊 ACCURACY")
print("Train:", accuracy_score(y[train_idx], train_pred))
print("Test:", accuracy_score(y[test_idx], test_pred))

print("\n📊 PRECISION")
print("Test:", precision_score(y[test_idx], test_pred))

print("\n📊 RECALL (IMPORTANT)")
print("Test:", recall_score(y[test_idx], test_pred))

print("\n📊 F1 SCORE")
print("Test:", f1_score(y[test_idx], test_pred))
```

```
import json

weights = w.tolist()  # convert numpy array to plain Python list

with open("ewsd_weights.json", "w") as f:
    json.dump({"weights": weights, "threshold": 0.42}, f)

print("Weights saved:", weights)
```

```
🌿 E.W.S.D MODEL RESULTS (EARLY WARNING SYSTEM)

📊 ACCURACY
Train: 0.7089552238805971
Test: 0.6875

📊 PRECISION
Test: 0.7045454545454546

📊 RECALL (IMPORTANT)
Test: 0.9393939393939394

📊 F1 SCORE
Test: 0.8051948051948052
Weights saved: [1.7706779696741606, 0.22992156129686284, 0.2886950301953366, 0.0014293330107278991, -1.3118442780865438, 0.322150643271176
```

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# ----------------------------
# LOAD DATA
# ----------------------------
data = pd.read_csv("plant_stress_data.csv")

if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({"No Rain": 0, "Rain": 1})

data = data.dropna()

X = data[["Temperature", "Humidity", "Soil Moisture", "Rain"]].values
y = data["Stress"].values

# normalization
X = (X - X.mean(axis=0)) / (X.std(axis=0) + 1e-6)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.15, random_state=42
)

# ----------------------------
# MODEL (FIXED SHAPES)
# ----------------------------
np.random.seed(42)

W1 = np.random.randn(4, 6) * 0.1
W2 = np.random.randn(6, 3) * 0.1
W3 = np.random.randn(3, 1) * 0.1

lr = 0.01

# ----------------------------
# ACTIVATIONS
# ----------------------------
def relu(x):
    return np.maximum(0, x)

def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -10, 10)))

# ----------------------------
# TRAINING
# ----------------------------
for epoch in range(200):

    for i in range(len(X_train)):

        x = X_train[i].reshape(1, -1)

        # forward
        z1 = relu(np.dot(x, W1))      # (1,6)
        z2 = relu(np.dot(z1, W2))     # (1,3)
        out = sigmoid(np.dot(z2, W3)) # (1,1)

        y_true = y_train[i]
```

```
        error = out - y_true

        # cost-sensitive learning (stress focus)
        if y_true == 1:
            error *= 2.0

        # ----------------------------
        # SIMPLE STABLE GRADIENTS
        # ----------------------------
        dW3 = np.dot(z2.T, error)
        dW2 = np.dot(z1.T, np.dot(error, W3.T))
        dW1 = np.dot(x.T, np.dot(error, W3.T).dot(W2.T))

        # update
        W3 -= lr * dW3
        W2 -= lr * dW2
        W1 -= lr * dW1

# ----------------------------
# PREDICTION
# ----------------------------
def predict(X):

    preds = []

    for x in X:

        x = x.reshape(1, -1)

        z1 = relu(np.dot(x, W1))
        z2 = relu(np.dot(z1, W2))
        out = sigmoid(np.dot(z2, W3))

        preds.append(1 if out > 0.5 else 0)

    return np.array(preds)

# ----------------------------
# RESULTS
# ----------------------------
train_pred = predict(X_train)
test_pred = predict(X_test)

print("\n🌿 T.A.S.N FIXED DEEP LEARNING MODEL")

print("\n📊 Accuracy")
print("Train:", accuracy_score(y_train, train_pred))
print("Test:", accuracy_score(y_test, test_pred))

print("\n📊 Precision")
print("Test:", precision_score(y_test, test_pred))

print("\n📊 Recall")
print("Test:", recall_score(y_test, test_pred))

print("\n📊 F1 Score")
print("Test:", f1_score(y_test, test_pred))
```

```
🌿 T.A.S.N FIXED DEEP LEARNING MODEL

📊 Accuracy
Train: 0.7164179104477612
Test: 0.7083333333333334

📊 Precision
Test: 0.6756756756756757

📊 Recall
Test: 0.9259259259259259

📊 F1 Score
Test: 0.78125
```

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# ----------------------------
# LOAD DATA
# ----------------------------
data = pd.read_csv("plant_stress_data.csv")

if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({"No Rain": 0, "Rain": 1})

data = data.dropna()
```

```
X = data[["Temperature", "Humidity", "Soil Moisture", "Rain"]].values
y = data["Stress"].values

# normalize
X = (X - X.mean(axis=0)) / (X.std(axis=0) + 1e-6)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.15, random_state=42
)

# ----------------------------
# MODEL PARAMETERS
# ----------------------------
np.random.seed(42)

W_att = np.random.randn(4) * 0.1     # attention weights
W_gate = np.random.randn(4) * 0.1    # gating weights
W_out = np.random.randn(4) * 0.1     # output weights

lr = 0.01

# ----------------------------
# ACTIVATIONS
# ----------------------------
def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -10, 10)))

def softmax(x):
    e = np.exp(x - np.max(x))
    return e / np.sum(e)

# ----------------------------
# FEATURE ENGINEERING (ENERGY IDEA)
# ----------------------------
def energy_features(x):

    T, H, S, R = x

    energy = np.array([
        abs(T),              # heat energy
        S * T,               # drought interaction energy
        (1 - S) * H,         # humidity stress
        R                    # rainfall shock
    ])

    return energy

# ----------------------------
# TRAINING
# ----------------------------
for epoch in range(180):

    for i in range(len(X_train)):

        x = X_train[i]

        e = energy_features(x)

        # -------------------------
        # ATTENTION MECHANISM
        # -------------------------
        att = softmax(e * W_att)

        context = e * att

        # -------------------------
        # GATING MECHANISM
        # -------------------------
        gate = sigmoid(np.dot(x, W_gate))

        # -------------------------
        # FINAL OUTPUT
        # -------------------------
        score = sigmoid(np.dot(context, W_out) * gate)

        error = score - y_train[i]

        # cost sensitive learning
        if y_train[i] == 1:
            error *= 2.0

        # -------------------------
        # GRADIENT UPDATE (SIMPLE)
        # -------------------------
        W_out -= lr * error * context
        W_att -= lr * error * e
        W_gate -= lr * error * x
```

```
# ----------------------------
# PREDICTION
# ----------------------------
def predict(X):

    preds = []

    for x in X:

        e = energy_features(x)

        att = softmax(e * W_att)
        context = e * att

        gate = sigmoid(np.dot(x, W_gate))

        score = sigmoid(np.dot(context, W_out) * gate)

        preds.append(1 if score > 0.5 else 0)

    return np.array(preds)

# ----------------------------
# RESULTS
# ----------------------------
train_pred = predict(X_train)
test_pred = predict(X_test)

print("\n🌿 G.A.T.E-NET MODEL RESULTS")

print("\n📊 Accuracy")
print("Train:", accuracy_score(y_train, train_pred))
print("Test:", accuracy_score(y_test, test_pred))

print("\n📊 Precision")
print("Test:", precision_score(y_test, test_pred))

print("\n📊 Recall")
print("Test:", recall_score(y_test, test_pred))

print("\n📊 F1 Score")
print("Test:", f1_score(y_test, test_pred))
```

```
🌿 G.A.T.E-NET MODEL RESULTS

📊 Accuracy
Train: 0.7201492537313433
Test: 0.5833333333333334

📊 Precision
Test: 0.574468085106383

📊 Recall
Test: 1.0

📊 F1 Score
Test: 0.7297297297297297
```

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# ----------------------------
# LOAD DATA
# ----------------------------
data = pd.read_csv("plant_stress_data.csv")

if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({"No Rain": 0, "Rain": 1})

data = data.dropna()

X = data[["Temperature", "Humidity", "Soil Moisture", "Rain"]].values
y = data["Stress"].values

# normalize
X = (X - X.mean(axis=0)) / (X.std(axis=0) + 1e-6)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.15, random_state=42
)

# ----------------------------
# MODEL PARAMETERS
# ----------------------------
np.random.seed(42)
```

```
w_main = np.random.randn(4) * 0.1   # main model
w_att = np.random.randn(4) * 0.1    # attention weights

lr = 0.01

# ----------------------------
# ACTIVATION
# ----------------------------
def sigmoid(x):
    return 1 / (np.exp(-np.clip(x, -10, 10)) + 1)

# ----------------------------
# ATTENTION FUNCTION
# ----------------------------
def attention(x, w_att):

    score = x * w_att
    score = np.exp(score) / np.sum(np.exp(score))

    return score

# ----------------------------
# TRAINING
# ----------------------------
for epoch in range(250):

    for i in range(len(X_train)):

        x = X_train[i]

        # attention weighted input
        att = attention(x, w_att)
        x_att = x * att

        # prediction
        pred = sigmoid(np.dot(x_att, w_main))

        error = pred - y_train[i]

        # COST SENSITIVE LEARNING
        if y_train[i] == 1:
            error *= 1.8   # emphasize stress detection

        # updates
        w_main -= lr * error * x_att
        w_att -= lr * error * x

# ----------------------------
# PREDICTION
# ----------------------------
def predict(X):

    preds = []

    for x in X:

        att = attention(x, w_att)
        x_att = x * att

        prob = sigmoid(np.dot(x_att, w_main))

        preds.append(1 if prob > 0.45 else 0)

    return np.array(preds)

# ----------------------------
# RESULTS
# ----------------------------
train_pred = predict(X_train)
test_pred = predict(X_test)

print("\n🌿 C.A.S.T MODEL RESULTS")

print("\n📊 Accuracy")
print("Train:", accuracy_score(y_train, train_pred))
print("Test:", accuracy_score(y_test, test_pred))

print("\n📊 Precision")
print("Test:", precision_score(y_test, test_pred))

print("\n📊 Recall")
print("Test:", recall_score(y_test, test_pred))

print("\n📊 F1 Score")
print("Test:", f1_score(y_test, test_pred))
```

```
🌿 C.A.S.T MODEL RESULTS
```

```
📊 Accuracy
Train: 0.753731343283582
Test: 0.6458333333333334

📊 Precision
Test: 0.6136363636363636

📊 Recall
Test: 1.0

📊 F1 Score
Test: 0.7605633802816901
```

```python
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# ----------------------------
# LOAD DATA
# ----------------------------
data = pd.read_csv("plant_stress_data.csv")

if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({"No Rain": 0, "Rain": 1})

data = data.dropna()

X = data[["Temperature", "Humidity", "Soil Moisture", "Rain"]].values
y = data["Stress"].values

# normalize
X = (X - X.mean(axis=0)) / (X.std(axis=0) + 1e-6)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.15, random_state=42
)

# ----------------------------
# MODEL PARAMETERS
# ----------------------------
np.random.seed(42)

w = np.random.randn(4) * 0.1
attention = np.ones(4)

lr = 0.01
momentum = 0.9
velocity = np.zeros(4)

# ----------------------------
# ACTIVATION
# ----------------------------
def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -10, 10)))

# ----------------------------
# ATTENTION FUNCTION
# ----------------------------
def compute_attention(x):

    # stress-sensitive attention
    score = np.abs(x)

    return score / (np.sum(score) + 1e-6)

# ----------------------------
# TRAINING (EPOCH LEARNING)
# ----------------------------
epochs = 500

for epoch in range(epochs):

    epoch_loss = 0

    # gradually increase stress importance
    stress_weight = 1 + (epoch / epochs)

    for i in range(len(X_train)):

        x = X_train[i]

        attention = compute_attention(x)
        x_att = x * attention

        pred = sigmoid(np.dot(w, x_att))

        error = pred - y_train[i]
```

```
        # cost-sensitive learning
        if y_train[i] == 1:
            error *= (1.5 * stress_weight)

        grad = error * x_att

        velocity = momentum * velocity + lr * grad
        w -= velocity


# ----------------------------
# PREDICTION (SMOOTHED OUTPUT)
# ----------------------------
def predict(X):

    preds = []

    for x in X:

        attention = compute_attention(x)
        x_att = x * attention

        prob = sigmoid(np.dot(w, x_att))

        # smoothing for stability
        preds.append(1 if prob > 0.48 else 0)

    return np.array(preds)

# ----------------------------
# RESULTS
# ----------------------------
train_pred = predict(X_train)
test_pred = predict(X_test)

print("\n🌿 E.L.S.A MODEL RESULTS")

print("\n📊 Accuracy")
print("Train:", accuracy_score(y_train, train_pred))
print("Test:", accuracy_score(y_test, test_pred))

print("\n📊 Precision")
print("Test:", precision_score(y_test, test_pred))

print("\n📊 Recall")
print("Test:", recall_score(y_test, test_pred))

print("\n📊 F1 Score")
print("Test:", f1_score(y_test, test_pred))

import pickle

with open("elsa_model.pkl", "wb") as f:
    pickle.dump({
        "weights": w
    }, f)

print("Model saved successfully!")
```

```
🌿 E.L.S.A MODEL RESULTS

📊 Accuracy
Train: 0.746268656716418
Test: 0.8125

📊 Precision
Test: 0.9090909090909091

📊 Recall
Test: 0.7407407407407407

📊 F1 Score
Test: 0.8163265306122449
Model saved successfully!
```

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier

# ----------------------------
# LOAD DATA
```

```
# ----------------------------
data = pd.read_csv("plant_stress_data.csv")

if data["Rain"].dtype == object:
    data["Rain"] = data["Rain"].map({"No Rain": 0, "Rain": 1})

data = data.dropna()

X = data[["Temperature", "Humidity", "Soil Moisture", "Rain"]].values
y = data["Stress"].values

# ----------------------------
# NORMALIZATION
# ----------------------------
scaler = StandardScaler()
X = scaler.fit_transform(X)

# ----------------------------
# TRAIN TEST SPLIT
# ----------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.15, random_state=42
)

# ----------------------------
# FUNCTION TO PRINT METRICS
# ----------------------------
def evaluate_model(name, y_test, y_pred):

    print(f"\n🌿 {name} RESULTS")

    print("\n📊 Accuracy:", accuracy_score(y_test, y_pred))
    print("📊 Precision:", precision_score(y_test, y_pred))
    print("📊 Recall:", recall_score(y_test, y_pred))
    print("📊 F1 Score:", f1_score(y_test, y_pred))

# ----------------------------
# 1️⃣ LOGISTIC REGRESSION
# ----------------------------
lr_model = LogisticRegression()
lr_model.fit(X_train, y_train)

lr_pred = lr_model.predict(X_test)
evaluate_model("Logistic Regression", y_test, lr_pred)

# ----------------------------
# 2️⃣ DECISION TREE
# ----------------------------
dt_model = DecisionTreeClassifier(max_depth=5)
dt_model.fit(X_train, y_train)

dt_pred = dt_model.predict(X_test)
```

```
🌿 Logistic Regression RESULTS

📊 Accuracy: 0.7708333333333334
📊 Precision: 0.7352941176470589
📊 Recall: 0.9259259259259259
📊 F1 Score: 0.819672131147541

🌿 Decision Tree RESULTS

📊 Accuracy: 1.0
📊 Precision: 1.0
📊 Recall: 1.0
📊 F1 Score: 1.0
```

Start coding or generate with AI.

Start coding or generate with AI.